Movie Magic–Smart Movie Ticket Booking System

Project Description:

With the increasing demand for a seamless and modern movie-watching experience, traditional ticket booking methods often fall short due to long queues, limited availability, and inconsistent service. To address this, the development team introduced **Movie Magic** - a smart, cloud-based movie ticket booking system. Built using Flask for backend development, hosted on AWS EC2, and integrated with DynamoDB for dynamic data management, the platform allows users to register, log in, and book movie tickets online with ease. Users can search for movies and events based on location, view real-time seat availability, and complete their bookings in just a few clicks. Upon booking, AWS SNS sends instant email notifications confirming ticket details, enhancing user engagement and trust. This cloud-native solution streamlines the entire movie ticketing process, ensuring fast, scalable, and user-friendly access to entertainment for all.

Scenario 1: Efficient Ticket Booking System for Users

In the Movie Magic System, AWS EC2 provides a reliable infrastructure capable of handling multiple users accessing the platform simultaneously. For example, a user can log in, navigate to the movie selection page, and seamlessly browse available shows and events in their city. They can then select a showtime, pick their preferred seats using an interactive layout, and confirm the booking—all in real-time. Flask manages backend processes, ensuring smooth data flow and quick response times even during high-traffic periods such as weekends or blockbuster releases.

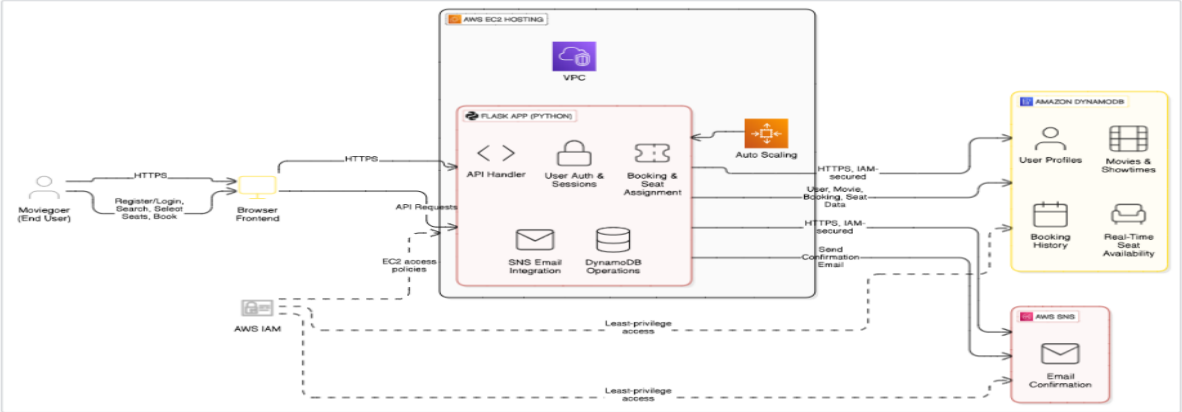
Scenario 2: Seamless Booking Confirmation Notifications

When a user completes a ticket booking, the Movie Magic System leverages AWS SNS to send instant email notifications to confirm the booking. For instance, once the booking is submitted, Flask processes the transaction, and SNS sends a customized email to the user with all ticket details, including movie name, date, time, and seat numbers. This real-time notification system enhances the customer experience and reduces uncertainty, while DynamoDB securely stores the booking records for both users and admins to manage and track.

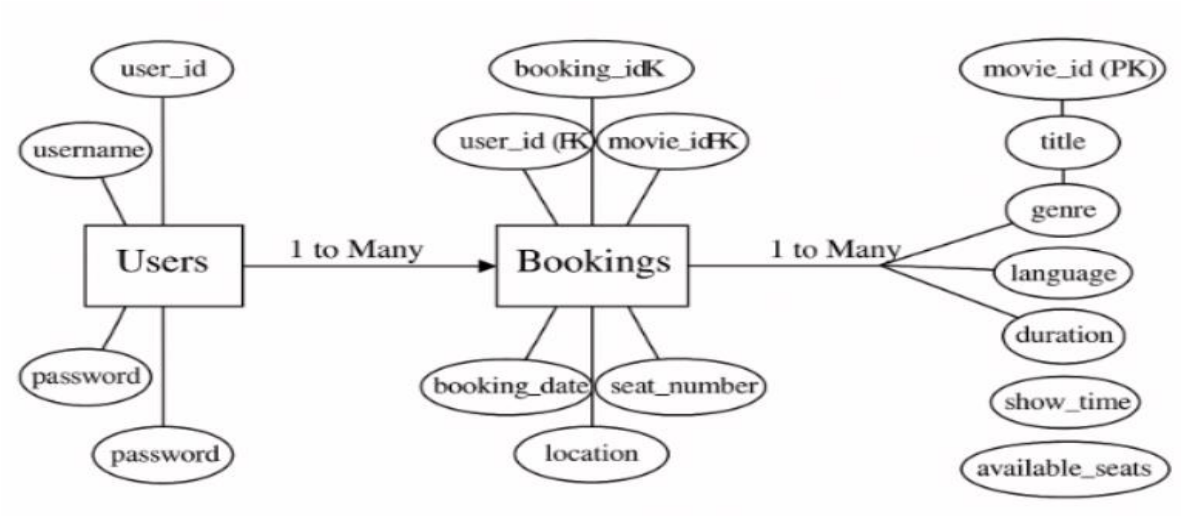
Scenario 3: Easy Access to Movies and Events

The Movie Magic platform offers users a seamless interface to explore currently running movies and upcoming live events. After logging in, a user can search by location or genre and instantly view listings with showtimes, ratings, and available seats. Flask dynamically fetches this data from DynamoDB, ensuring real-time updates on seat availability and event information. Meanwhile, the EC2-hosted application remains stable and responsive, even during traffic spikes, providing users with an uninterrupted and enjoyable booking experience.

AWS ARCHITECTURE



Entity Relationship (ER)Diagram:



Pre-requisites:

1. **AWS Account Setup**: AWS Account Setup
2. **Understanding IAM**: IAM Overview
3. **Amazon EC2 Basics**: EC2 Tutorial
4. **DynamoDB Basics**: DynamoDB Introduction
5. **SNS Overview**: SNS Documentation
6. **Git Version Control**: Git Documentation

Project Workflow

1. **AWS Account Setup and Login**
   Activity 1.1: Create an AWS account.
   Activity 1.2: Log in to the AWS Management Console.

2. **DynamoDB Database Creation and Setup**
   Activity 2.1: Create DynamoDB tables.
   Activity 2.2: Configure attributes for users, movies, and bookings.

3. **SNS Notification Setup**
   Activity 3.1: Create an SNS topic for ticket notifications.
   Activity 3.2: Subscribe an email for confirmation alerts.

4. **Backend Development and Application Setup**
   Activity 4.1: Develop backend using Flask (Python).
   Activity 4.2: Integrate AWS services using boto3.

5. **IAM Role Setup**
   Activity 5.1: Create an IAM role.
   Activity 5.2: Attach DynamoDB and SNS policies.

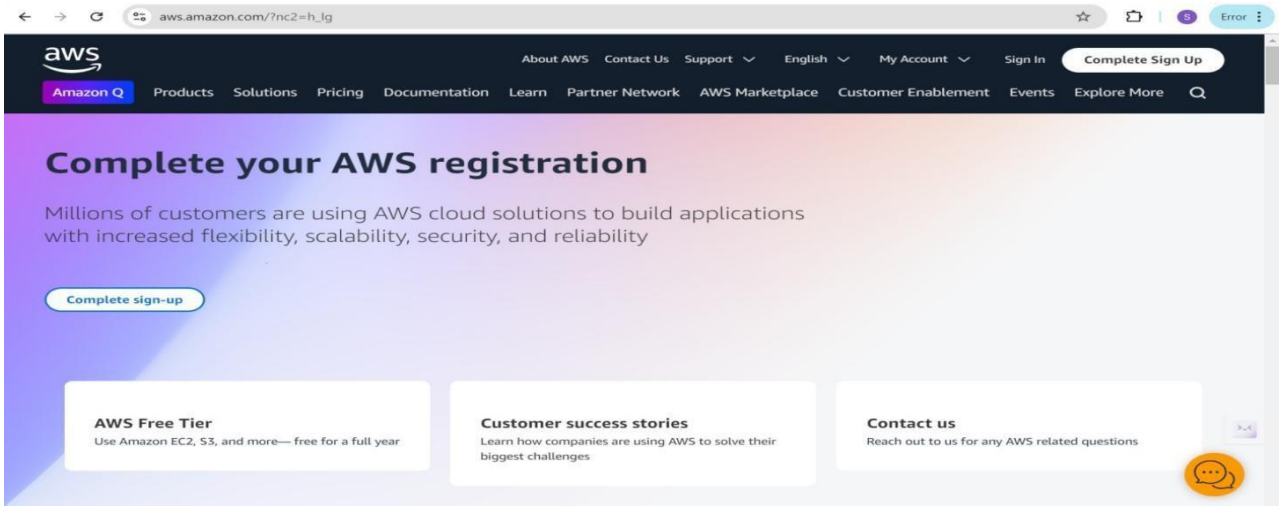
6. **EC2 Instance Setup**
   Activity 6.1: Launch an EC2 instance.
   Activity 6.2: Configure security groups for HTTP and SSH.

7. **Deployment on EC2**
   Activity 7.1: Clone or upload project to EC2.
   Activity 7.2: Install dependencies and run Flask app.

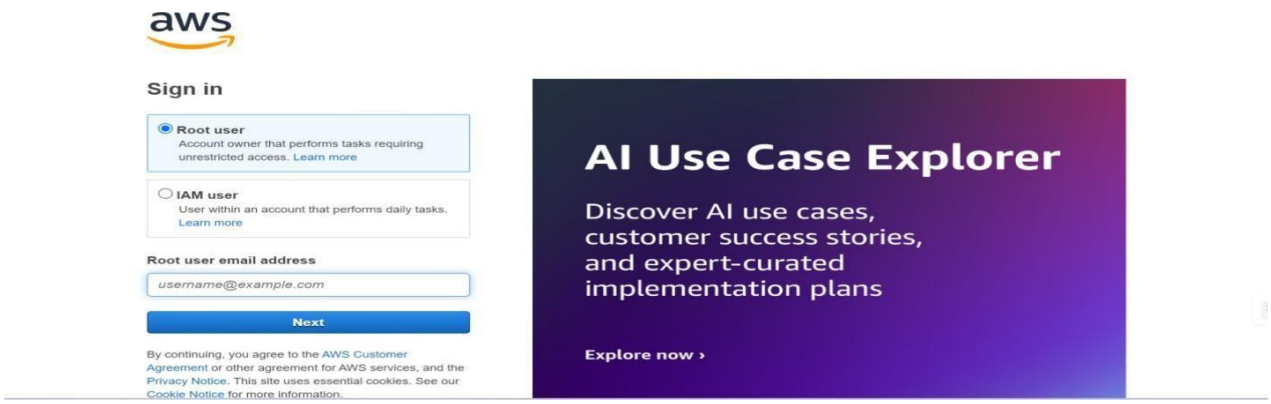
8. Testing and Deployment
Activity 8.1: Test registration, login, movie browsing, booking, admin management, and notifications.

Milestone 1: AWS Account Setup and Login

- **Activity 1.1: Set up an AWS account if not already done.**
  - Sign up for an AWS account and configure billing settings.



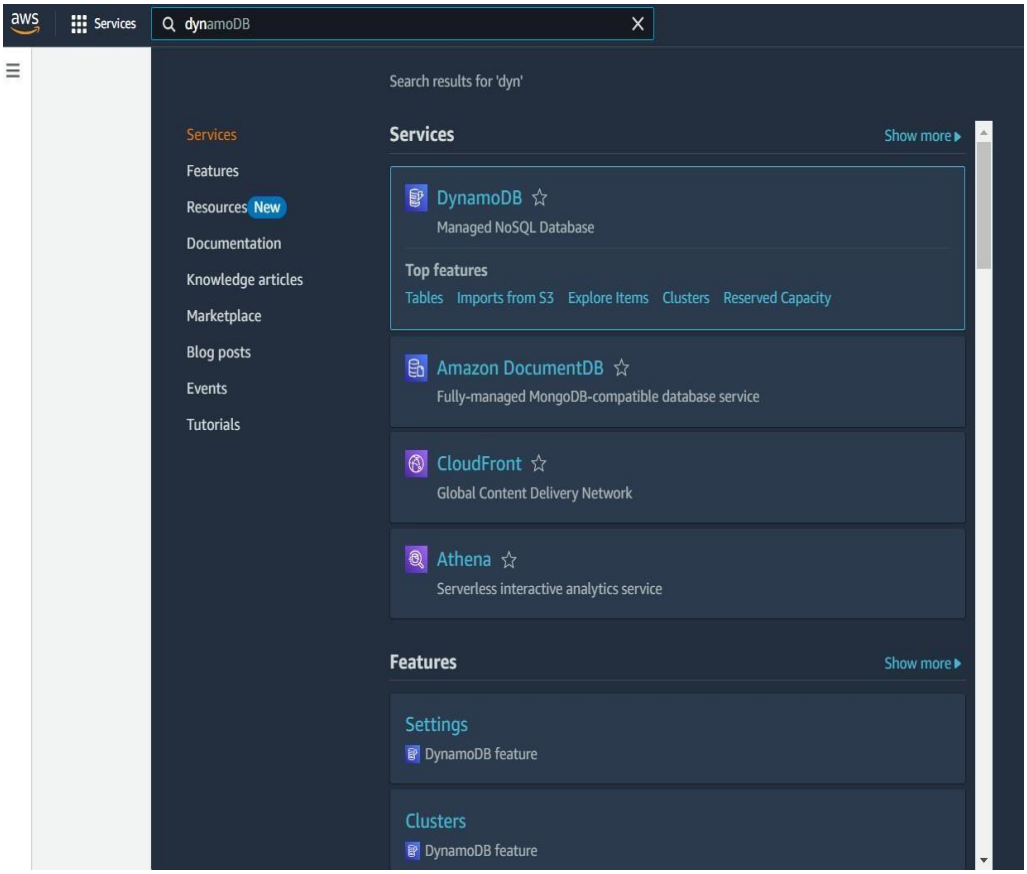
- **Activity 1.2: Log in to the AWS Management Console**
  - After setting up your account, log in to the AWS Management Console.

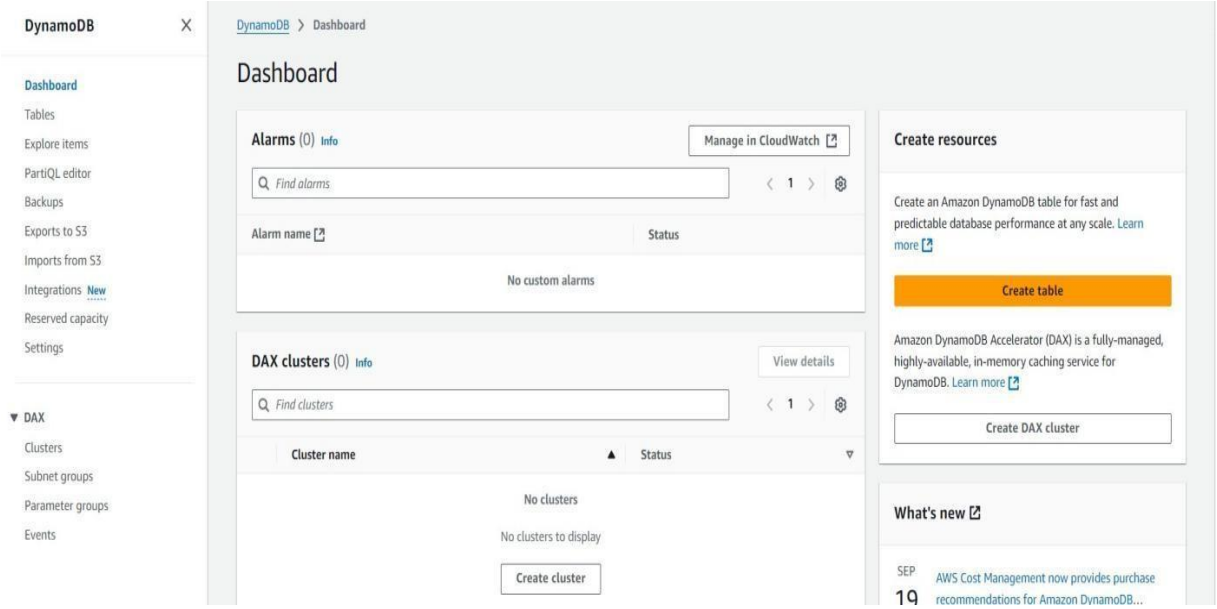


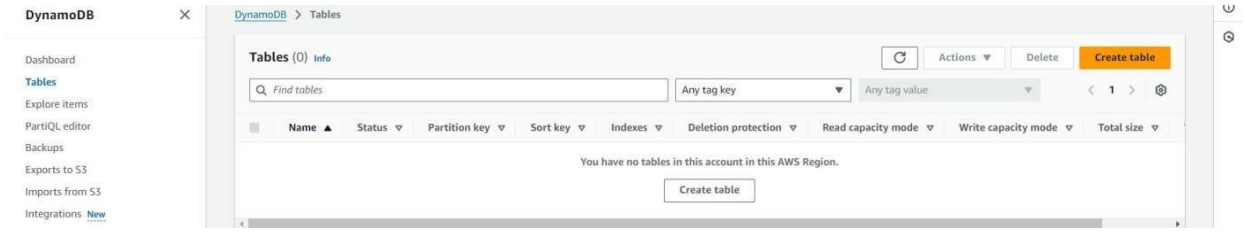
Milestone 2: DynamoDB Database Creation and Setup

- **Activity 2.1:Navigate to the DynamoDB**

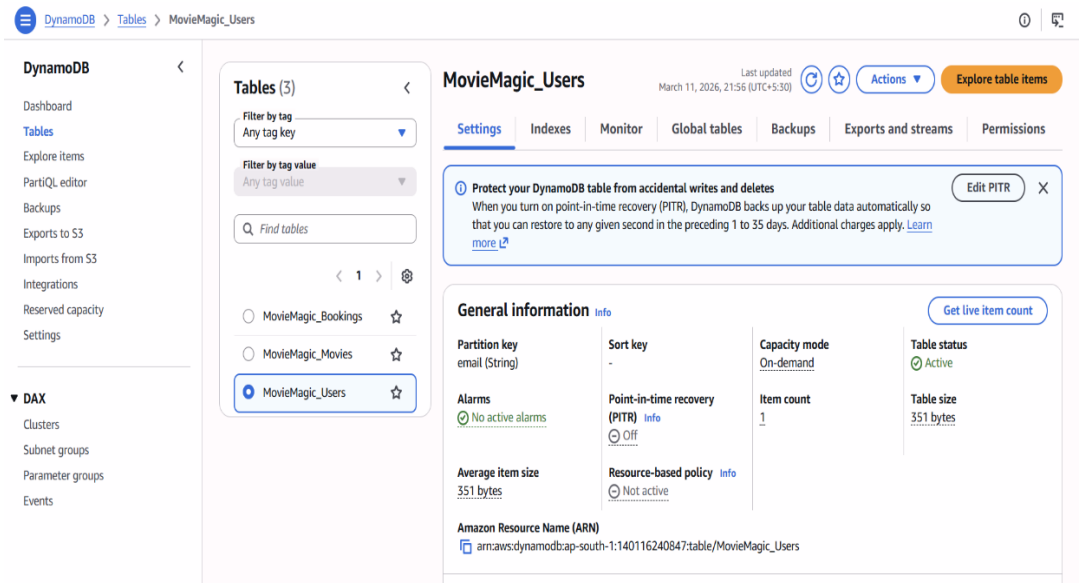
- In the AWS Console, navigate to DynamoDB and click on create tables.



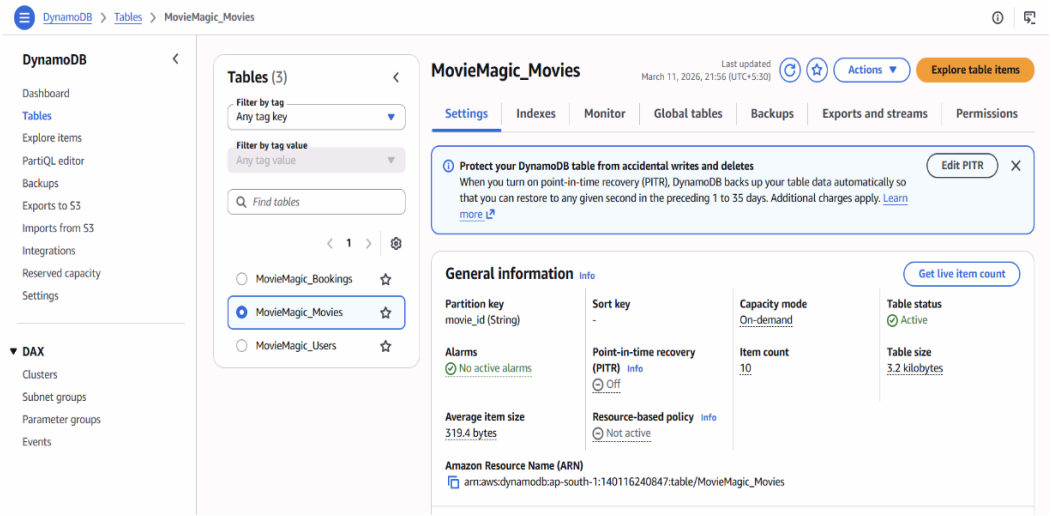




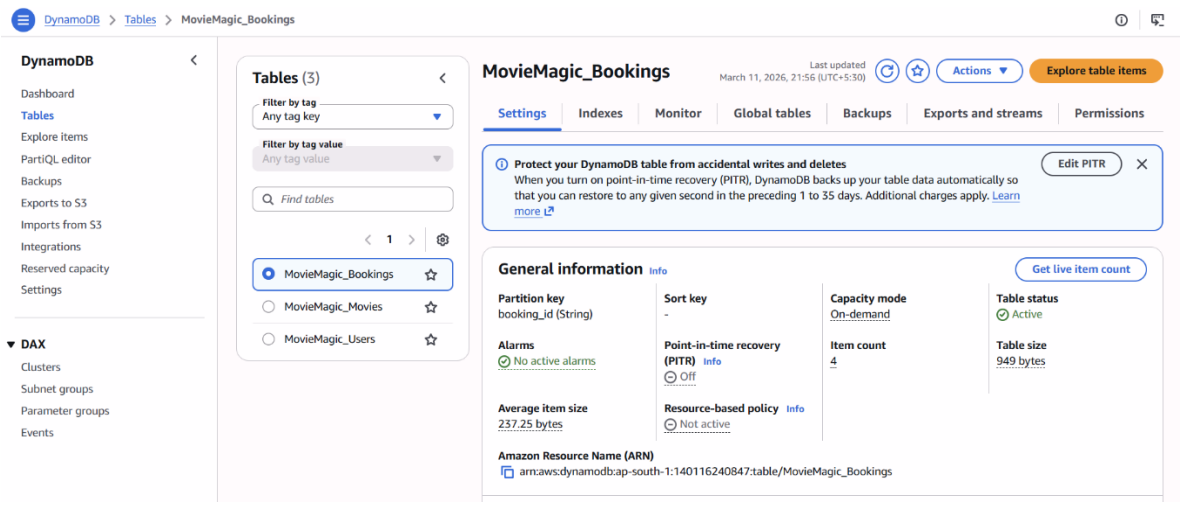
- **Activity 2.2:Create DynamoDB tables for storing registration details and booking records.**
  - Create MovieMagic_users for storing registered user details with partition key "Email" with type String and click on create tables.

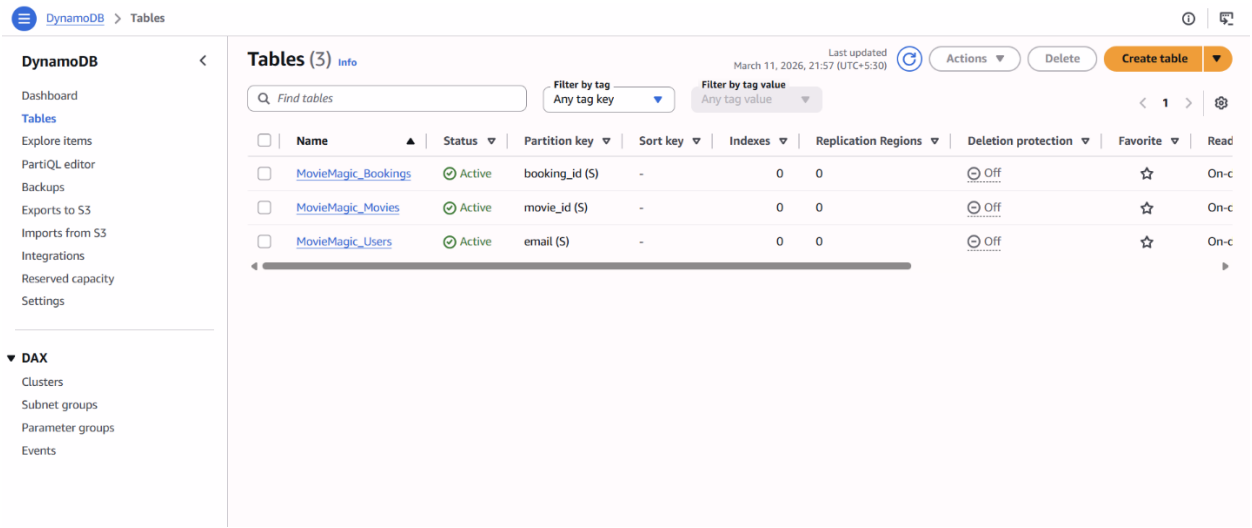


Follow the same steps to create a MovieMagic_Movies with partition key "movie_id" with type String and click on create tables.



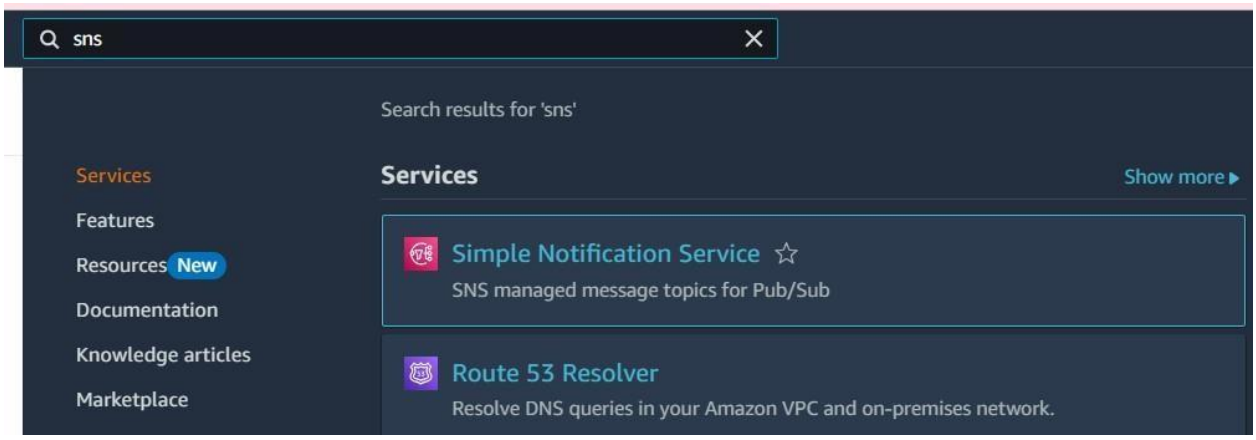
Follow the same steps to create a Moviemagic_Bookings with partition key “booking_id” with type String and click on create tables.

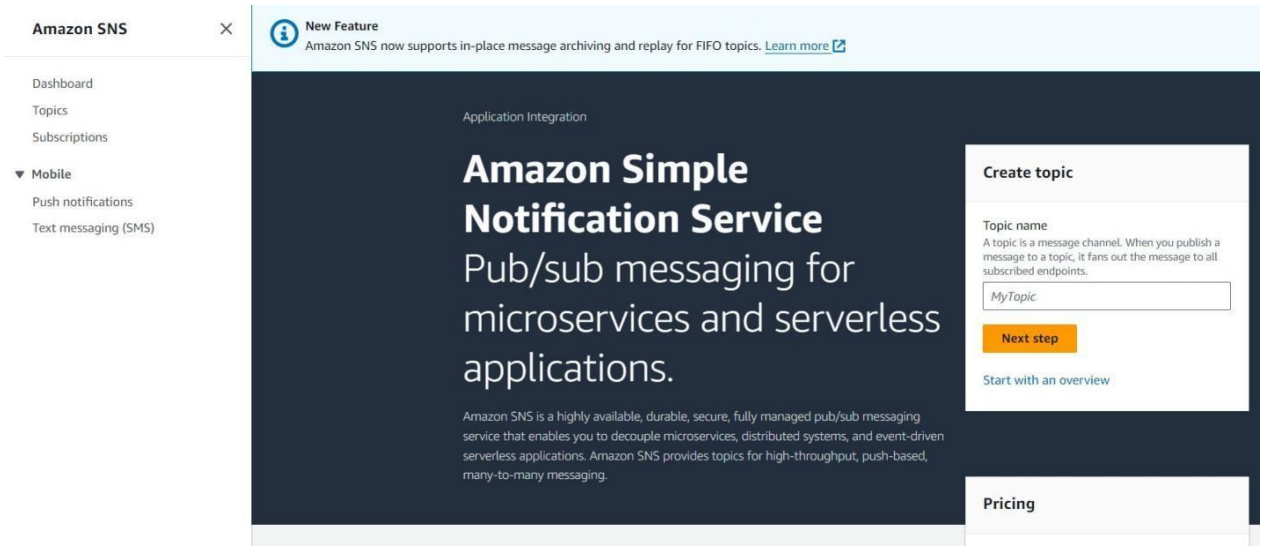




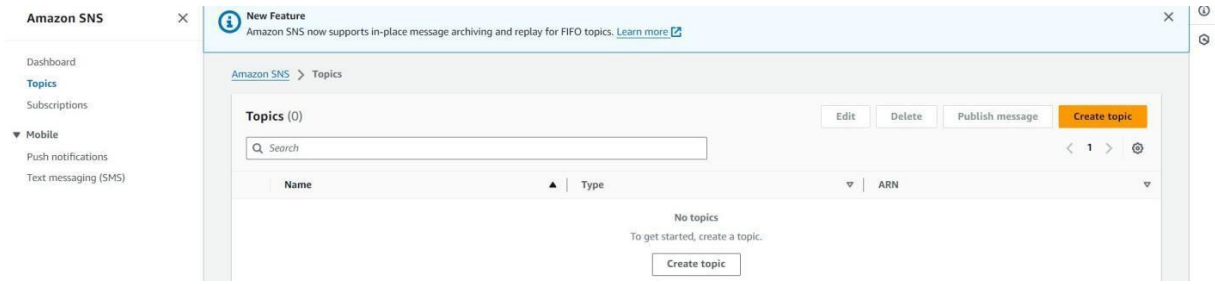
Milestone 3: SNS Notification Setup

- **Activity 3.1:** Create an SNS Topic named MovieTicketNotificationsfor sending notifications for users after booking.
    - In the AWS Console, search for SNS and navigate to the SNS Dashboard.

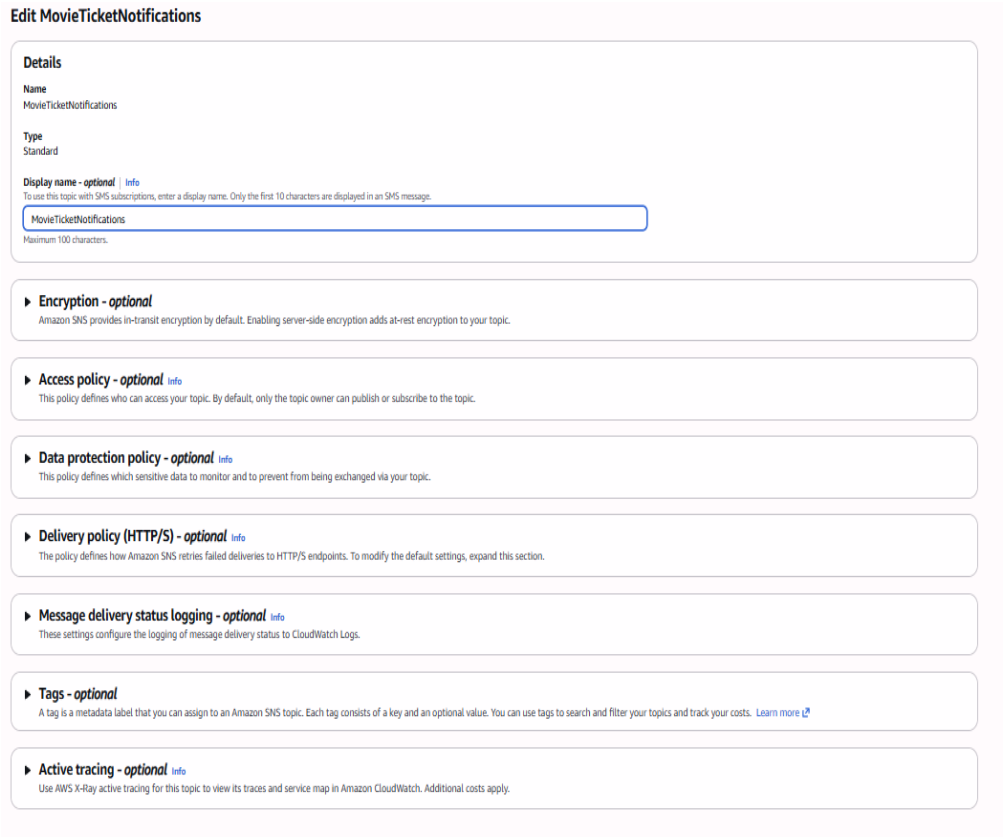




- Click on **Create Topic** and choose a name for the topic.



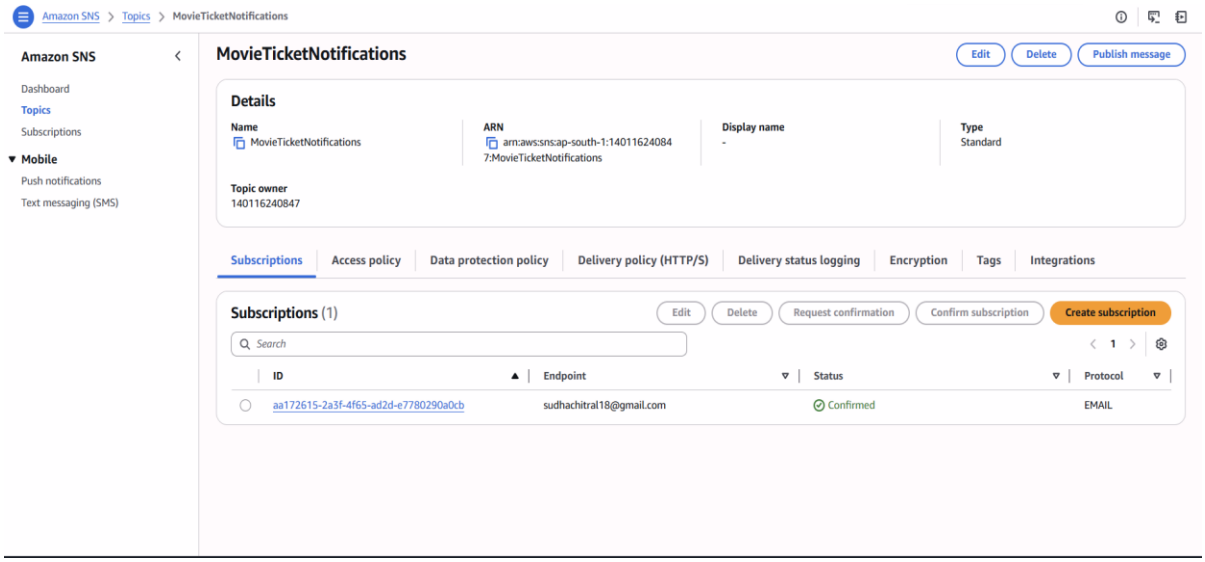
- Choose Standard type for general notification use cases and Click on Create Topic.



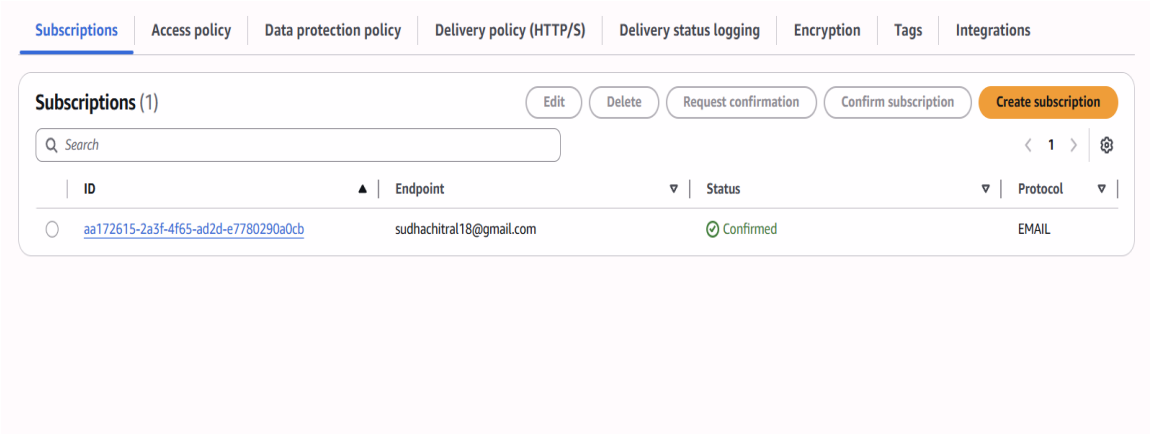
- Configure the SNS topic and note down the **Topic ARN**.

- **Activity 3.2: Subscribe email to the topic to receive booking and cancellation alerts.**

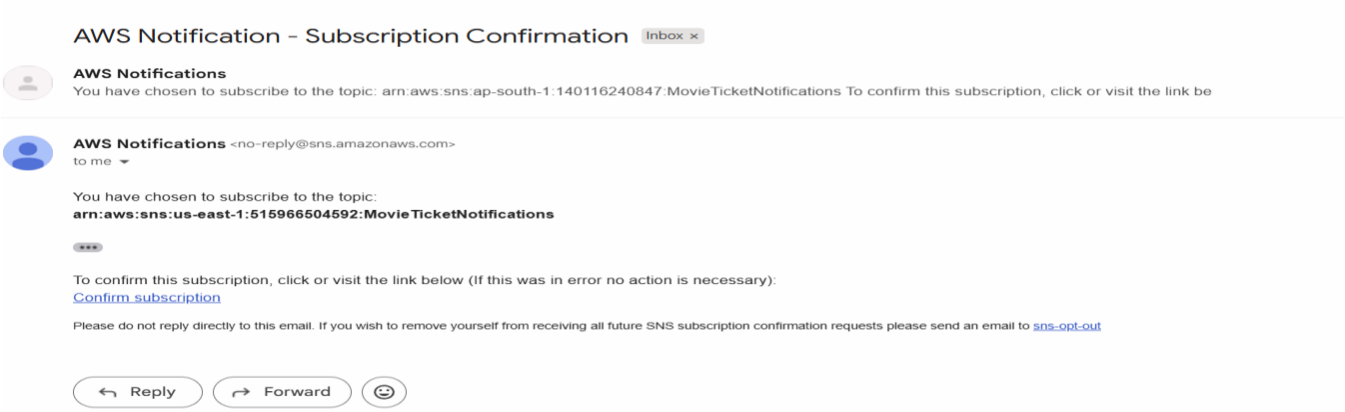
  - Subscribe patients or doctors to the SNS topic via email so they receive notifications when appointments are booked or diagnosis reports are submitted.

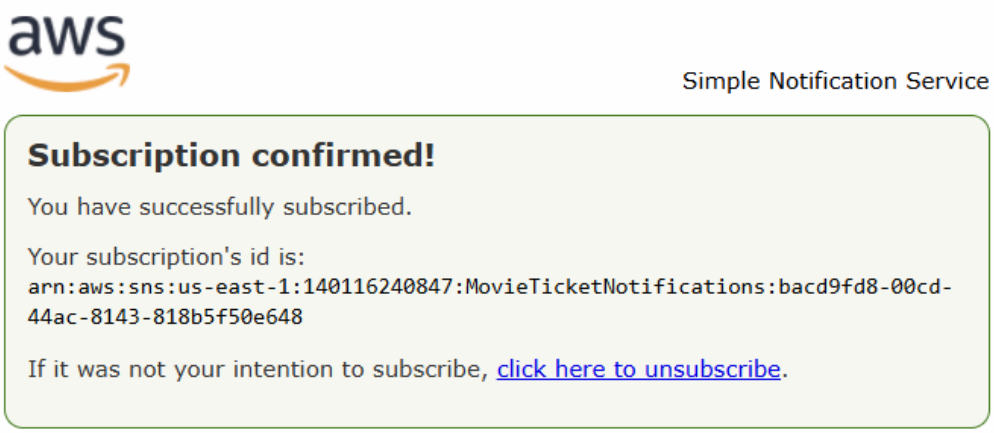


  - After subscription request for the mail confirmation



Navigate to the subscribed Email account and Click on the confirm subscription in the AWS Notification- Subscription Confirmation mail.





Successfully done with the SNS mail subscription and setup, now store the ARN link.

Milestone 4:Backend Development and Application Setup

- **Activity 4.1: Develop the backend using Flask**
- File Structure:

```
movie-magic
|
├── app.py
├── templates
|   ├── index.html
|   ├── login.html
|   ├── signup.html
|   ├── dashboard.html
|   ├── movie_details.html
|   ├── booking.html
|   ├── payment.html
|   ├── confirmation.html
|   ├── profile.html
|   └── admin.html
|
├── static
|   ├── css
|   ├── js
|   └── images
```

Description: Create a templates folder with HTML files for registration, login, appointment booking, diagnosis submission, and dashboard pages, while handling backend logic in app.py and utils/data.py.

Description of the code :

- **Flask App Initialization**

  Import Libraries:

```
from flask import Flask, render_template, request, redirect, url_for, session, flash
from werkzeug.security import generate_password_hash, check_password_hash
from datetime import datetime
import boto3
from boto3.dynamodb.conditions import Attr
from decimal import Decimal
import uuid
import os
from botocore.exceptions import ClientError
```

Description: Import Flask modules for routing and templates, Werkzeug for password security, Boto3 for AWS DynamoDB and SNS integration, UUID for generating unique IDs, and supporting libraries like datetime, Decimal, and os for configuration and data handling.

```
app = Flask(__name__)

# Security & Config
app.secret_key = 'your_static_secret_key_here'
AWS_REGION = os.environ.get('AWS_REGION', 'us-east-1')
SNS_TOPIC_ARN = 'arn:aws:sns:ap-south-1:977099000730:MovieTicketNotifications'
```

Initialize flask app:

```
app = Flask(__name__)
```

Description: Initialize the Flask application by creating an instance of the `Flask` class with the current module name.

DynamoDB Connection:

```
# AWS Services
dynamodb = boto3.resource('dynamodb', region_name=AWS_REGION)
sns = boto3.client('sns', region_name=AWS_REGION)
users_table = dynamodb.Table('MovieMagic_Users')
bookings_table = dynamodb.Table('MovieMagic_Bookings')
movies_table = dynamodb.Table('MovieMagic_Movies')
```

Description: Use boto3 to connect to DynamoDB to manage patient information, appointment details, and diagnosis records, and specify the region_name of the DynamoDB tables.

SNS Connection:

```
sns = boto3.client('sns', region_name=AWS_REGION)
```

Description: Configure **SNS** to send notifications when a booking request is submitted

Routes for Web Pages

Register User:

```
@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        # Get user input from form
        name = request.form['name']
        email = request.form['email']
        password = request.form['password']

        # Hash the password for security
        hashed_password = generate_password_hash(password)
```

Description: Collecting registration data, and stores user details in the database

login Route (GET/POST):

```
@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        email = request.form['email']
        password = request.form['password']

        # Admin Login Check
        if email == "admin@moviemagic.com" and password == "admin123":
            session['user'] = {'name': 'Administrator', 'email': email, 'is_admin': True, 'theme': 'dark'}
            return redirect(url_for('admin_dashboard'))

        try:
            response = users_table.get_item(Key={'email': email})
            if 'Item' in response:
                user = response['Item']
                if check_password_hash(user['password'], password):
                    session['user'] = {
                        'name': user['name'],
                        'email': user['email'],
                        'theme': user.get('theme', 'dark') # Load User Theme
                    }
                    return redirect(url_for('dashboard'))
            flash('Invalid credentials', 'danger')
        except ClientError:
            flash('Login error', 'danger')
    return render_template('login.html')
```

Description: Verifies user credentials, increments login count, and redirects to the dashboard on success.

Dashboard route:

```
@app.route('/dashboard')
def dashboard():
    if 'user' not in session: return redirect(url_for('login'))
    try:
        response = movies_table.scan()
        movies = replace_decimals(response.get('Items', []))
    except ClientError as e:
        movies = []
    return render_template('dashboard.html', movies=movies)
```

Description: Define the /dashboard route to display the user's appointment and medical data from DynamoDB after verifying the session and fetching user details using their email.

Index route:

```
@app.route('/')
def index():
    return render_template('index.html')
```

Description:This route checks the user's login status and automatically redirects logged-in users to their role-based dashboard, while new or logged-out users are shown the homepage.

Dashboard route:

```
@app.route('/dashboard')
def dashboard():
    # Check if user is logged in
    if 'user' not in session:
        return redirect(url_for('login'))

    try:
        # Retrieve movies from DynamoDB
        response = movies_table.scan()
        movies = replace_decimals(response.get('Items', []))

    except ClientError as e:
        # Handle database error
        print(e)
        movies = []

    # Render dashboard page with movie data
    return render_template('dashboard.html', movies=movies)
```

Description: The code retrieves the list of available movies from the AWS DynamoDB MovieMagic_Movies table and displays them on the user dashboard. It converts DynamoDB Decimal values into normal numeric values for proper display in templates. If the user is not logged in, the system redirects them to the login page. Proper error handling ensures the application continues running even if there is a database access issue.

Movie_details Route:

```
@app.route('/movie/<movie_id>')
def movie_details(movie_id):
    if 'user' not in session: return redirect(url_for('login'))
    try:
        response = movies_table.get_item(Key={'movie_id': movie_id})
        movie = replace_decimals(response.get('Item'))
        if not movie:
            flash('Movie not found', 'danger')
            return redirect(url_for('dashboard'))
        return render_template('movie_details.html', movie=movie)
    except ClientError:
        flash('Error loading movie', 'danger')
        return redirect(url_for('dashboard'))
```

Description: This route checks if the user is logged in before allowing access to movie details. It retrieves the selected movie information from the **MovieMagic_Movies** table in DynamoDB using the movie ID. The system converts DynamoDB Decimal values into normal numeric values for display. Finally, the movie data is sent to the **movie_details.html** template, and error handling ensures smooth execution if any issue occurs.

Booking Route:

```
@app.route('/booking')
def booking():
    if 'user' not in session: return redirect(url_for('login'))
    return render_template('booking.html',
                           movie=request.args.get('movie'),
                           theater=request.args.get('theater'),
                           address=request.args.get('address'),
                           price=request.args.get('price'))
```

Description: This route checks whether the user is logged in before accessing the booking page. If the user is not authenticated, the system redirects them to the login page. It retrieves movie, theater, address, and price details from the request parameters and sends them to the **booking.html** template to display the booking form.

Payment Route:

```
@app.route('/payment', methods=['POST'])
def payment():
    if 'user' not in session: return redirect(url_for('login'))
    return render_template('payment.html', booking_details=request.form)
```

Description: This route checks whether the user is logged in before processing the payment page. If the user is not authenticated, the system redirects them to the login page. It receives booking details from the booking form and passes them to the **payment.html** template to display the payment information before confirming the booking.

Confirm Booking Route:

```
@app.route('/confirm_booking', methods=['POST'])
def confirm_booking():
    if 'user' not in session: return redirect(url_for('login'))
    data = request.form
    try:
        booking_id = f"MM-{datetime.now().strftime('%Y%m%d')}-{str(uuid.uuid4())[:8]}"
        fake_payment_id = f"PAY-{str(uuid.uuid4())[:12].upper()}"

        booking_item = {
            'booking_id': booking_id,
            'movie_name': data.get('movie'),
            'theater': data.get('theater'),
            'date': data.get('date'),
            'time': data.get('time'),
            'seats': data.get('seats'),
            'amount_paid': data.get('amount'),
            'address': data.get('address'),
            'booked_by': session['user']['email'],
            'user_name': session['user']['name'],
            'payment_id': fake_payment_id,
            'booking_time': datetime.now().isoformat()
        }

        bookings_table.put_item(Item=booking_item)
        send_email(booking_item)
        return render_template('confirmation.html', booking=booking_item)
    except Exception as e:
        print(e)
        flash('Booking failed.', 'danger')
        return redirect(url_for('dashboard'))
```

Description: This route verifies that the user is logged in before confirming the booking. It generates a unique booking ID and payment ID, then stores the booking details in the **MovieMagic_Bookings** table in DynamoDB. After saving the booking, the system sends a confirmation email using **AWS SNS** and displays the booking confirmation on the **confirmation.html** page.

Profile route:

```
@app.route('/profile')
def profile():
    if 'user' not in session: return redirect(url_for('login'))
    user_email = session['user']['email']
    user_bookings = []
    user_info = {}
    try:
        user_response = users_table.get_item(Key={'email': user_email})
        if 'Item' in user_response:
            user_info = user_response['Item']
            # Ensure theme key exists for template
            if 'theme' not in user_info:
                user_info['theme'] = 'dark'
        else:
            user_info = session['user']

        response = bookings_table.scan(FilterExpression=Attr('booked_by').eq(user_email))
        user_bookings = replace_decimals(response.get('Items', []))
    except ClientError:
        pass
    return render_template('profile.html', user=user_info, bookings=user_bookings)
```

Description: This route checks if the user is logged in before accessing the profile page. It retrieves the user's information from the **MovieMagic_Users** table and their booking history from the **MovieMagic_Bookings** table in DynamoDB. The system converts Decimal values to normal numbers and displays the user details and bookings in the **profile.html** template.

Admin Dashboard route:

```
@app.route('/admin')
def admin_dashboard():
    if 'user' not in session or not session.get('user', {}).get('is_admin'):
        return redirect(url_for('login'))
    try:
        response = movies_table.scan()
        movies = replace_decimals(response.get('Items', []))
    except ClientError:
        movies = []
    return render_template('admin.html', movies=movies)
```

Description: This route verifies that the logged-in user has **admin privileges** before allowing access. It retrieves all movie records from the **MovieMagic_Movies** table in DynamoDB. The movie data is then displayed in the **admin.html** template, where the admin can manage movies such as adding, editing, or deleting them.

Logout route:

```
@app.route('/logout')
def logout():
    session.pop('user', None)
    return redirect(url_for('index'))
```

Description: This route logs the user out of the system by removing the user information from the session. After clearing the session data, the system redirects the user to the **home page (index.html)** to ensure secure logout.

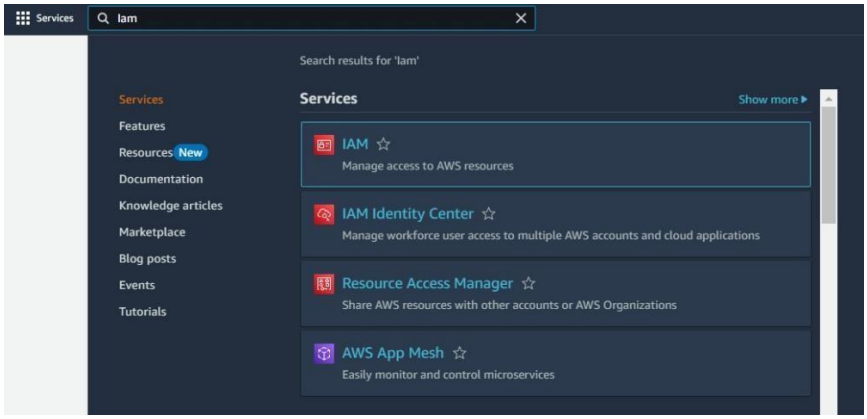
Deployment Code:

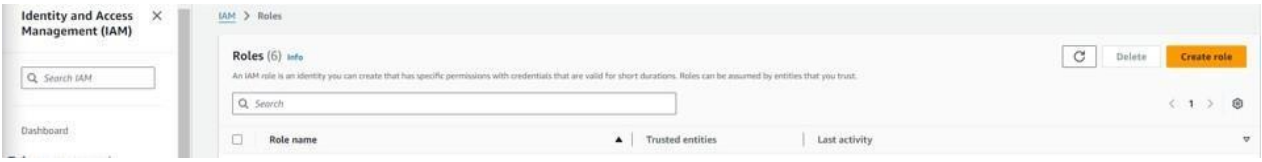
```
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)
```

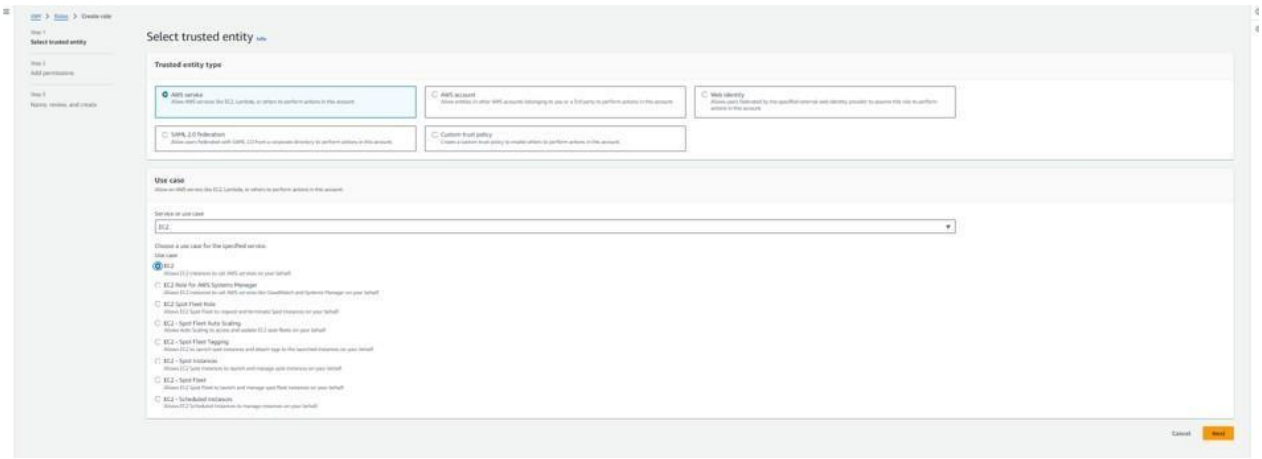
Description:
Start the Flask application on **all network interfaces (0.0.0.0)** at **port 5000** with **debug mode enabled**. This allows the application to be accessed through the EC2 public IP and helps developers easily identify and fix errors during development and testing.

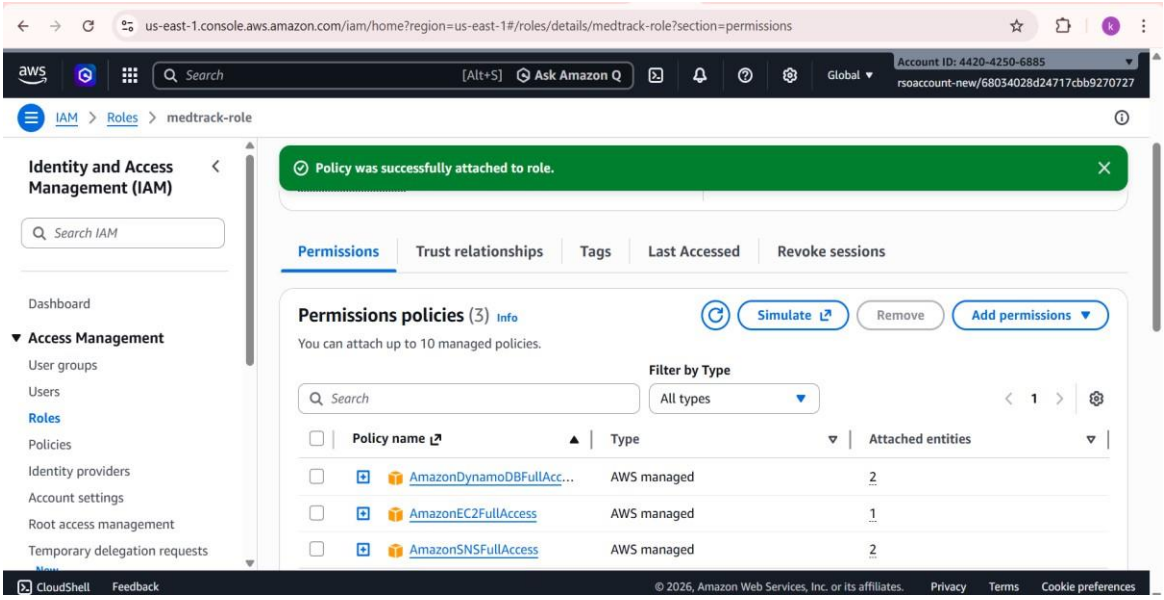
Milestone 5: IAM Role Setup

- **Activity 5.1:Create IAM Role.**
    - In the AWS Console, go to IAM and create a new IAM Role for EC2 to interact with DynamoDB and SNS.







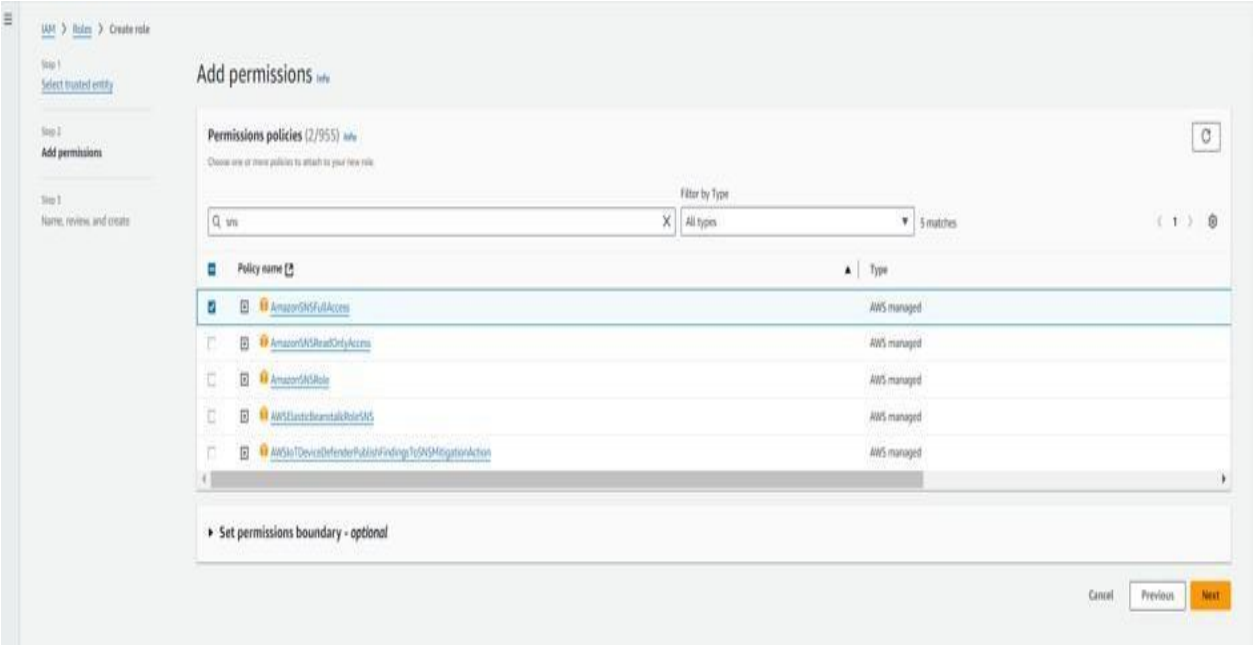


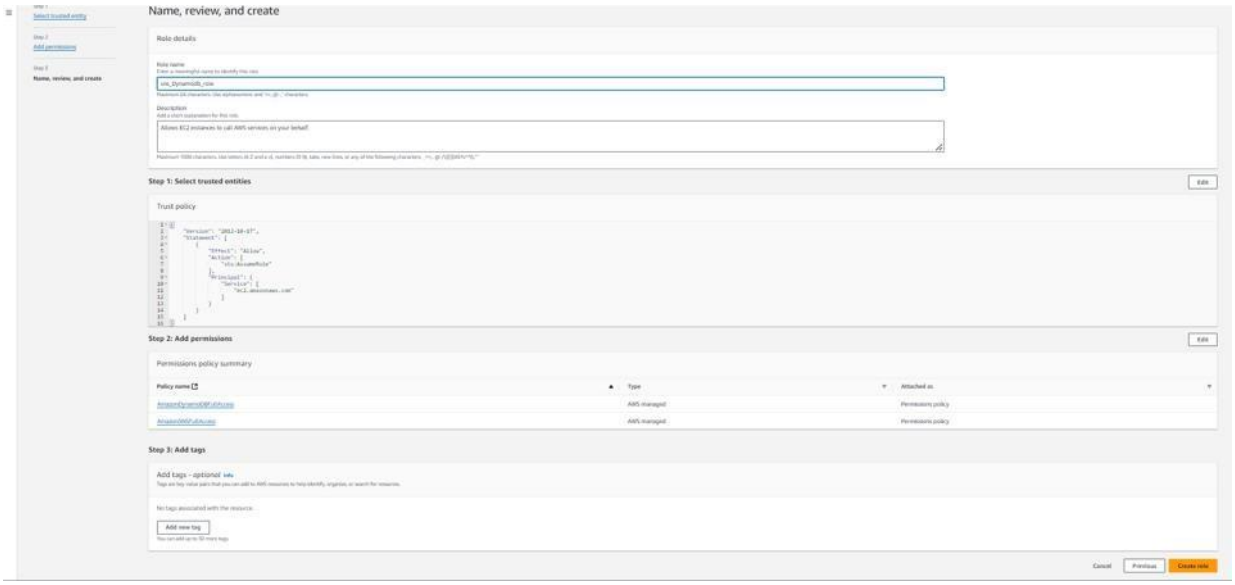
- **Activity 5.2: Attach Policies.**

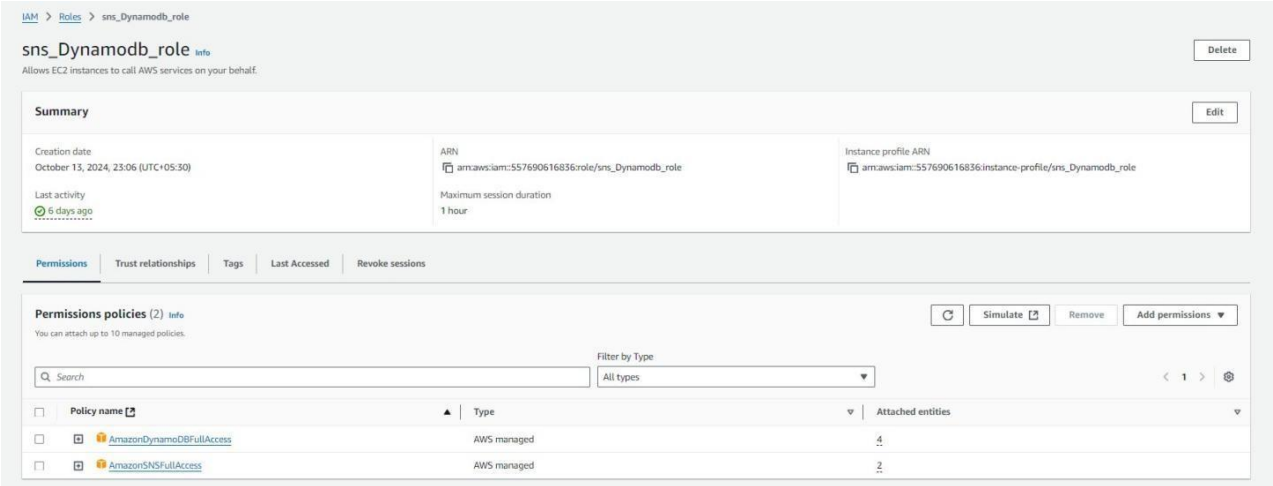
  Attach the following policies to the role:

  - AmazonDynamoDBFullAccess: Allows EC2 to perform read/write operations on DynamoDB.
  - AmazonSNSFullAccess: Grants EC2 the ability to send notifications via SNS.

AmazonEC2FullAcess:Allows full access to manage Amazon EC2 resources such as launching, stopping, and configuring EC2 instance

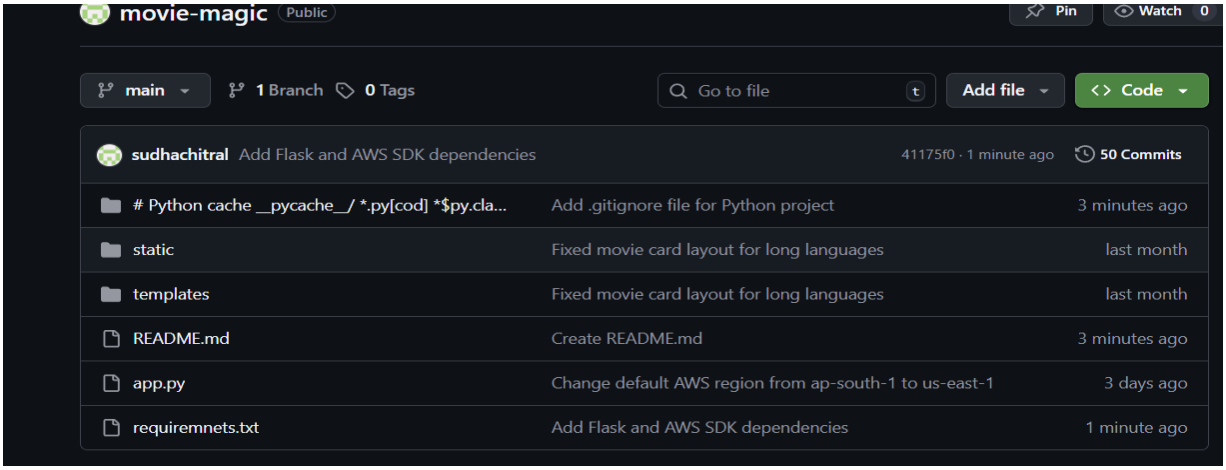

IAM > Roles > Create role
Step 1
Select trusted entity
Step 2
Add permissions
Step 3
Name, review, and create
Add permissions Info
Permissions policies (2/955) Info
Choose one or more policies to attach to your new role.
Filter by Type
sns
All types
5 matches
Policy name
Type
AmazonSNSFullAccess
AWS managed
AmazonSNSReadOnlyAccess
AWS managed
AmazonSNSRole
AWS managed
AWSElasticBeanstalkRoleSNS
AWS managed
AWSIoTDeviceDefenderPublishFindingsToSNSMitigationAction
AWS managed
Set permissions boundary - optional
Cancel
Previous
Next

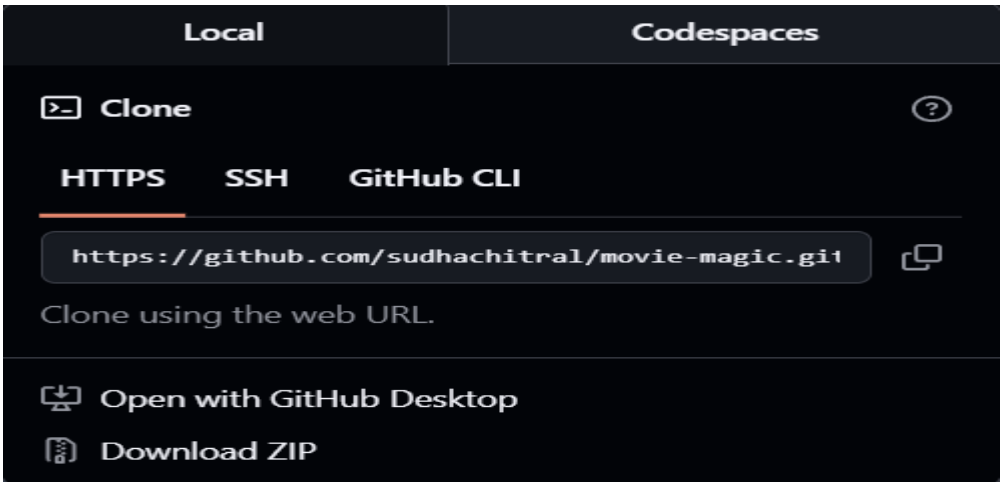

Name, review, and create
Role details
Step 1: Select trusted entities
Trust policy
Step 2: Add permissions
Permissions policy summary
Policy name
Type
Attached as
Step 3: Add tags
Add tags - optional Info
Add new tag
Cancel
Previous
Create role


IAM > Roles > sns_Dynamodb_role
sns_Dynamodb_role Info
Allows EC2 instances to call AWS services on your behalf.
Delete
Summary
Edit
Creation date
October 13, 2024, 23:06 (UTC+05:30)
ARN
arn:aws:iam::557690616836:role/sns_Dynamodb_role
Instance profile ARN
arn:aws:iam::557690616836:instance-profile/sns_Dynamodb_role
Last activity
6 days ago
Maximum session duration
1 hour
Permissions
Trust relationships
Tags
Last Accessed
Revoke sessions
Permissions policies (2) Info
You can attach up to 10 managed policies.
Simulate
Remove
Add permissions
Filter by Type
Search
All types
Policy name
Type
Attached entities
AmazonDynamoDBFullAccess
AWS managed
4
AmazonSNSFullAccess
AWS managed
2

Milestone 6: EC2 Instance Setup

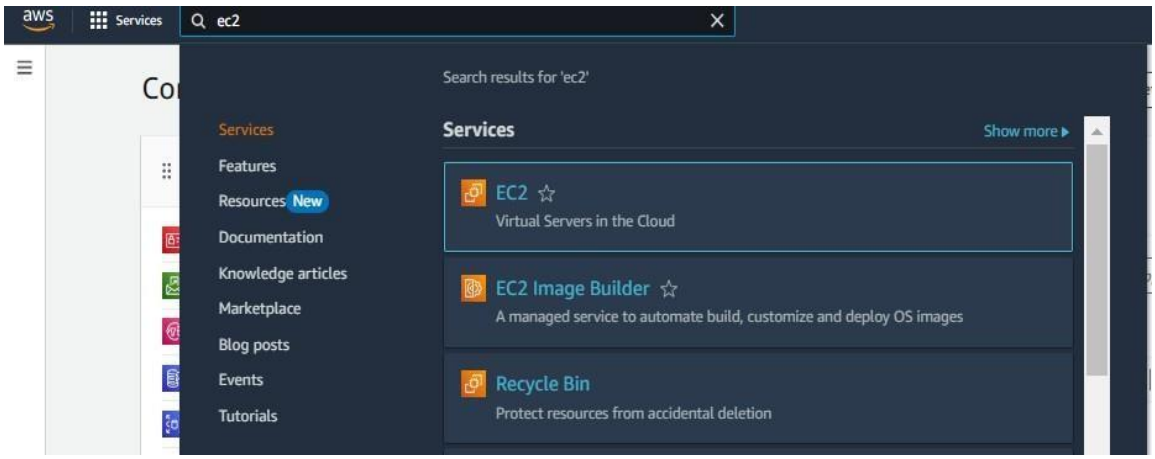
- **Note: Load your Flask app and Html files into GitHub repository.**



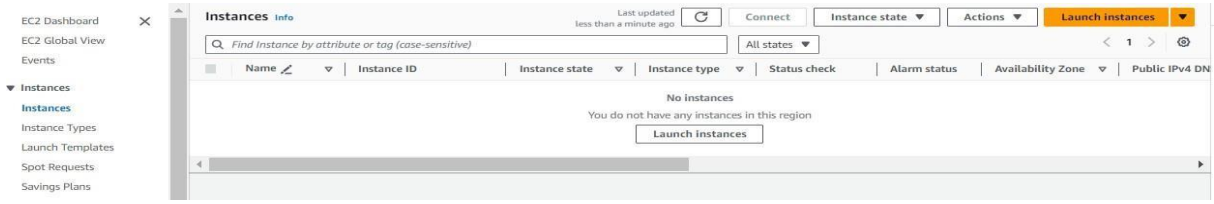


- **Activity 6.1: Launch an EC2 instance to host the Flask application.**

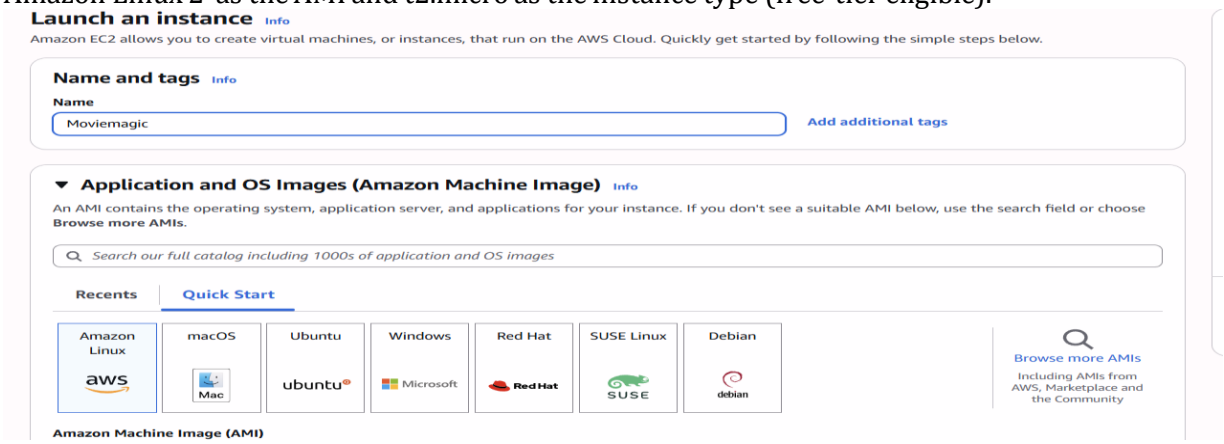
  - **Launch EC2 Instance**
    - In the AWS Console, navigate to EC2 and launch a new instance.



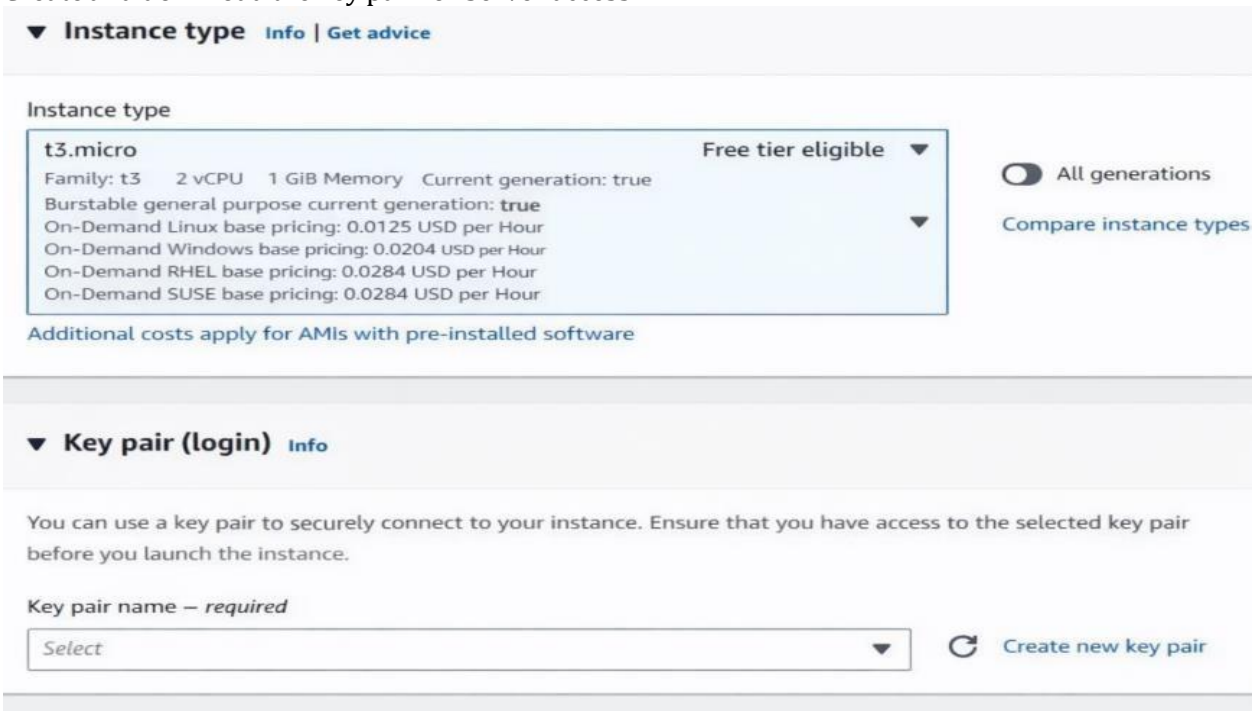
- Click on Launch instance to launch EC2 instance



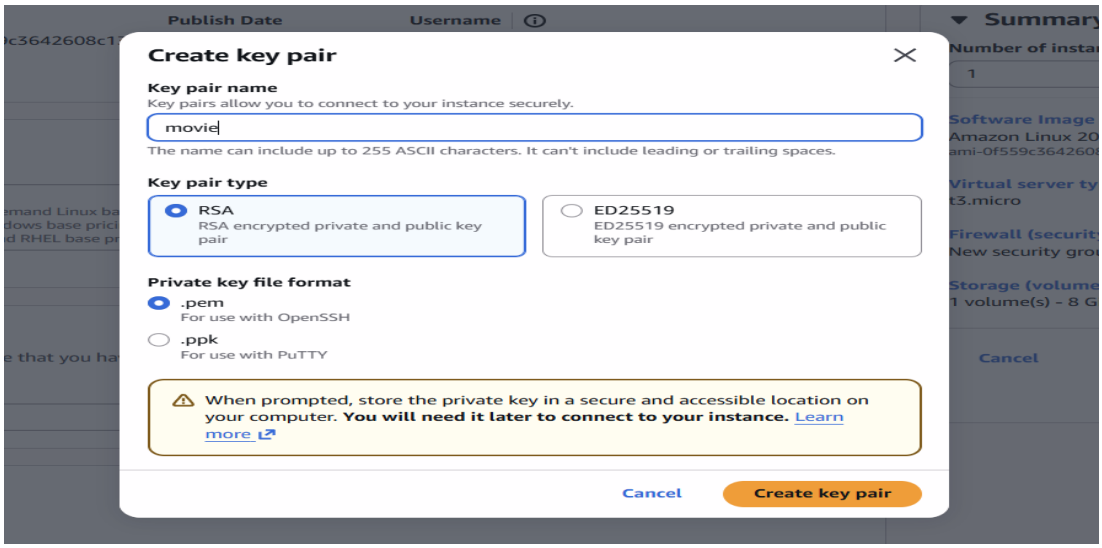
Choose Amazon Linux 2 as the AMI and t2.micro as the instance type (free-tier eligible).



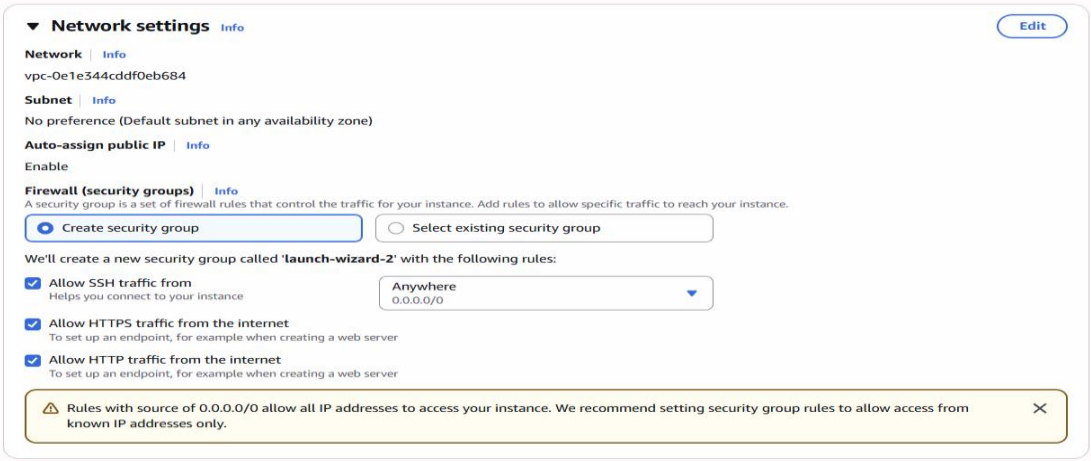
- Create and download the key pair for Server access.

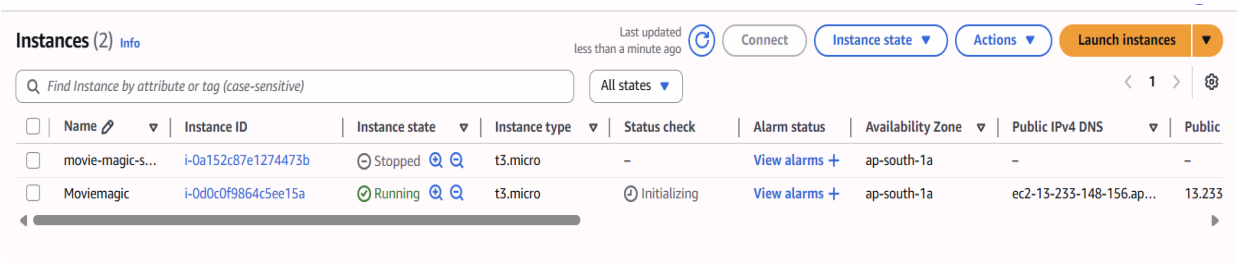


-

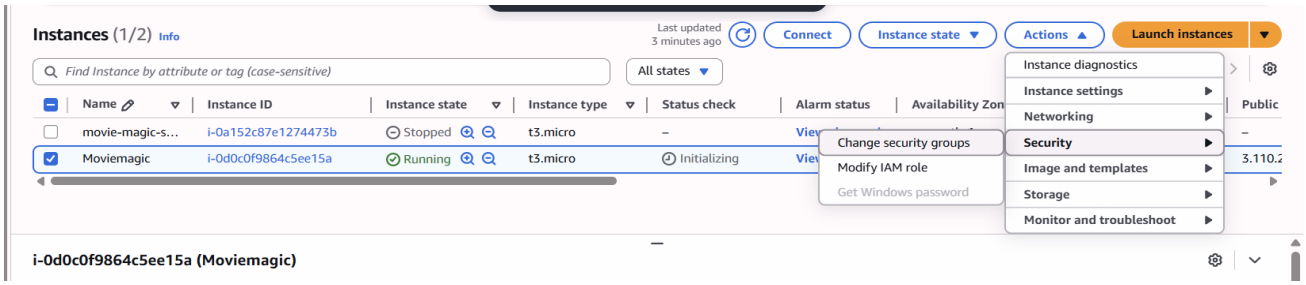


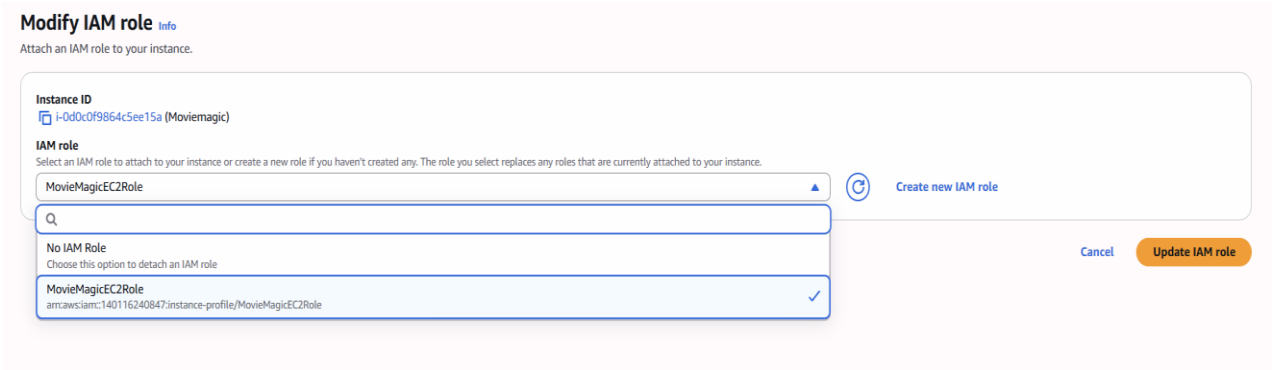
- **Activity 6.2:Configure security groups for HTTP, and SSH access.**





- To connect to EC2 using **EC2 Instance Connect**, start by ensuring that an **IAM role** is attached to your EC2 instance. You can do this by selecting your instance, clicking on **Actions**, then navigating to **Security** and selecting **Modify IAM Role** to attach the appropriate role. After the IAM role is connected, navigate to the **EC2** section in the **AWS Management Console**. Select the **EC2 instance** you wish to connect to. At the top of the **EC2 Dashboard**, click the **Connect** button. From the connection methods presented, choose **EC2 Instance Connect**. Finally, click **Connect** again, and a new browser-based terminal will open, allowing you to access your EC2 instance directly from your browser.






```
* Debug mode: on
ARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.31.76.8:5000
ress CTRL+C to quit
* Restarting with stat
home/ec2-user/.local/lib/python3.9/site-packages/boto3/compat.py:89: PythonDeprecationWarning: Boto3 will no longer support Python 3.9 starting April 29, 2026. To con
ue receiving service updates, bug fixes, and security updates please upgrade to Python 3.10 or later. More information can be found here: https://aws.amazon.com/blogs
veloper/python-support-policy-updates-for-aws-sdks-and-tools/
 warnings.warn(warning, PythonDeprecationWarning)
* Debugger is active!
* Debugger PIN: 124-177-569
```

Milestone 7: Deployment on EC2

Activity 7.1: Install Software on the EC2 Instance

 # Update and install Python + pip

sudo apt update

sudo apt install python3-pip -y

sudo apt install git -y

sudo apt install python3-venv -y

python3 -m venv venv

source venv/bin/activate

Activity 7.2:Clone Your Flask Project from GitHub

Clone your project repository from GitHub into the EC2 instance using Git.

Run: 'git clone https://github.com/your-github-username/your-repository-name.git'

Note: change your-github-username and your-repository-name with your credentials

here: 'git clone https://github.com/K-Shreelekha/aws-cloud-healthcare-system.git'

- This will download your project to the EC2 instance.

To navigate to the project directory, run the following command:

cd <your-folder>

Once inside the project directory, configure and run the Flask application by executing the following command with elevated privileges:

Run the Flask Application

git pull origin main (Only if you made changes in git and want to reflect them in the EC2 terminal.)

 # Install requirements

pip install flask boto3

Launch the Flask app:

sudo -E venv/bin/python3 app.py

Verify the Flask app is running:

http://your-ec2-public-ip

- Run the Flask app on the EC2 instance

```
* Debug mode: on
ARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.31.76.8:5000
ress CTRL+C to quit
* Restarting with stat
home/ec2-user/.local/lib/python3.9/site-packages/boto3/compat.py:89: PythonDeprecationWarning: Boto3 will no longer support Python 3.9 starting April 29, 2026. To con
ue receiving service updates, bug fixes, and security updates please upgrade to Python 3.10 or later. More information can be found here: https://aws.amazon.com/blogs
veloper/python-support-policy-updates-for-aws-sdks-and-tools/
 warnings.warn(warning, PythonDeprecationWarning)
* Debugger is active!
* Debugger PIN: 124-177-569
```

Access the website through:

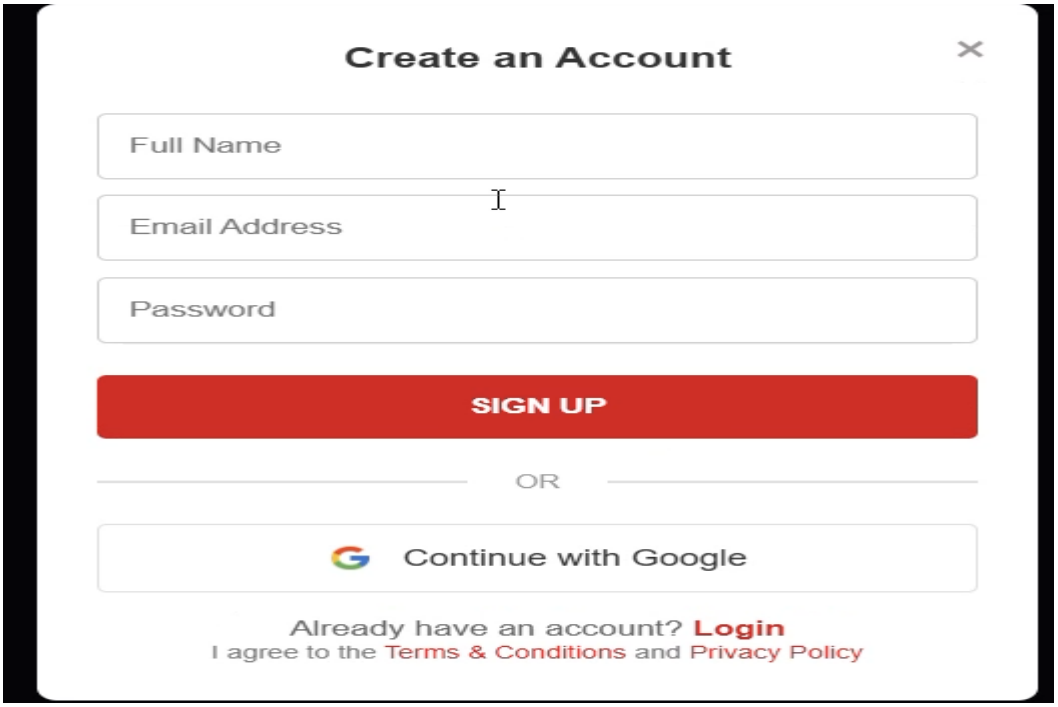
PublicIPs: http://3.236.79.91:5000/

Milestone 8: Testing and Deployment

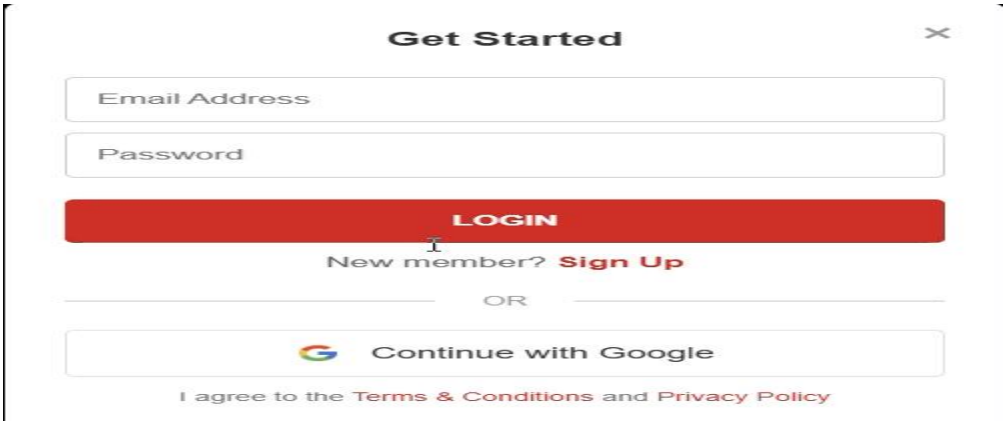
- **Activity 8.1: Conduct functional testing to verify user registration, login, book requests, and notifications.**

Home Page:

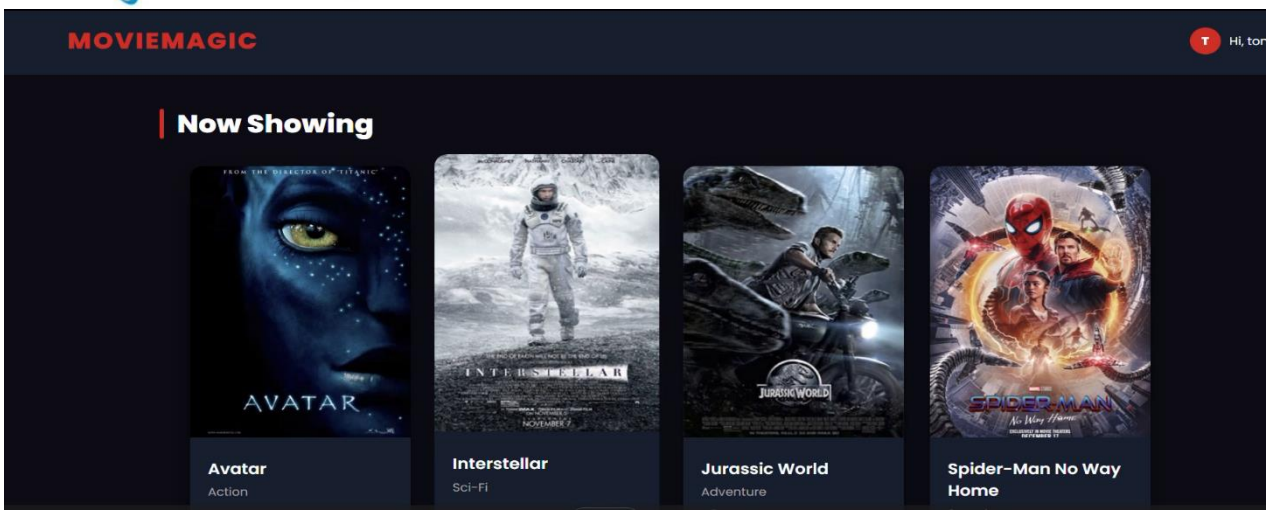
Registration Page:



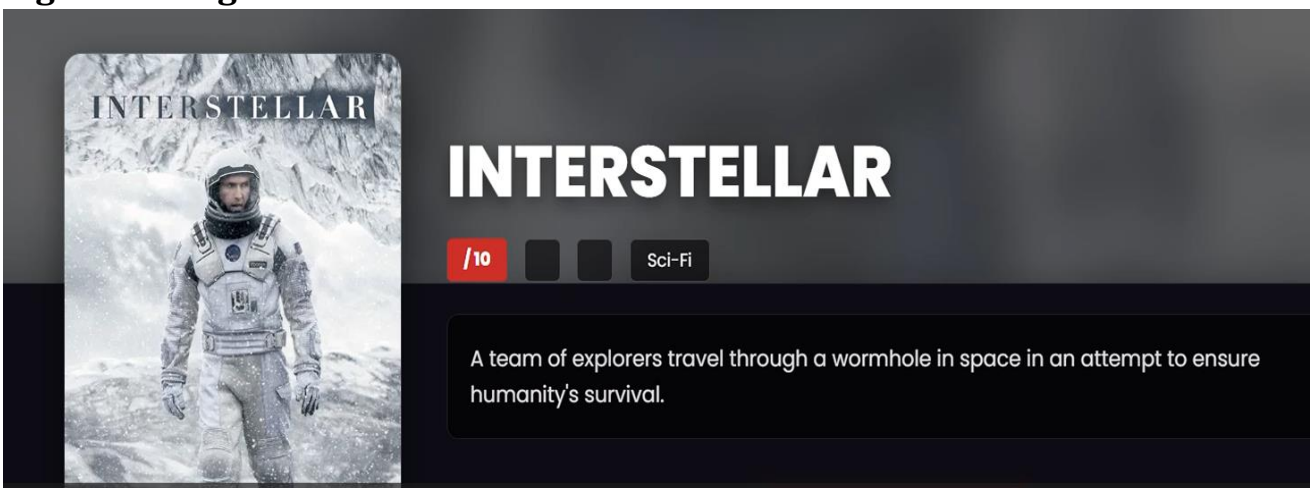
Login Page:

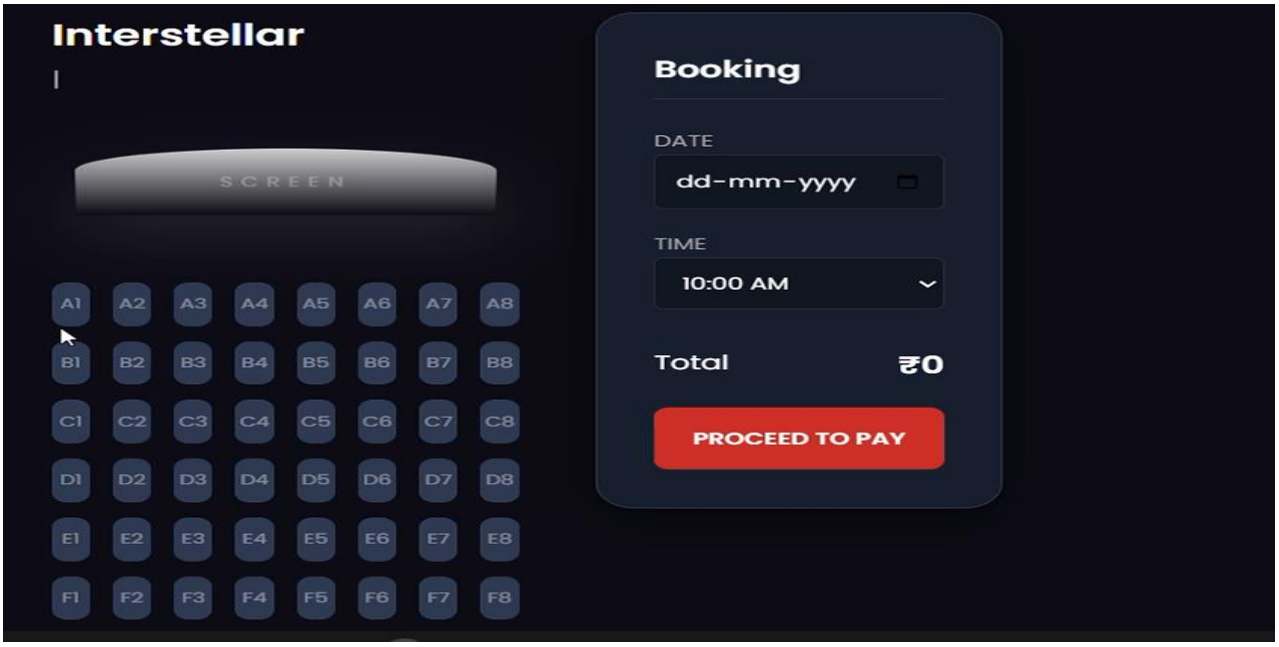


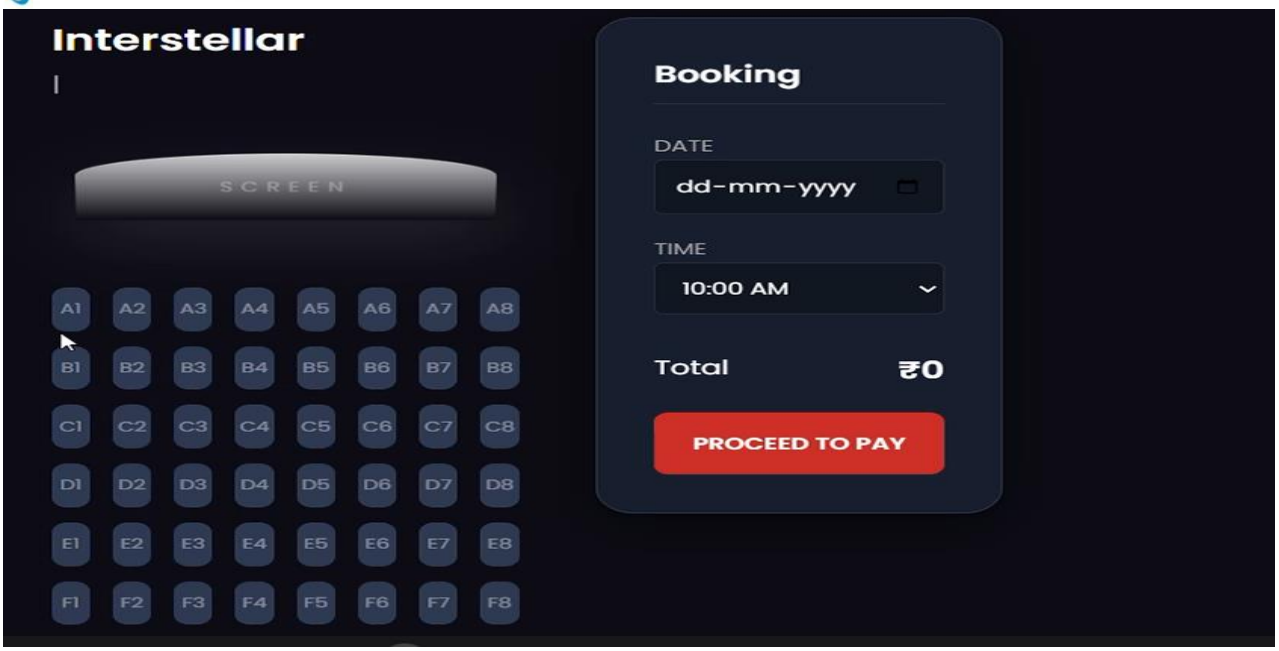
Movie Dashboard Page:



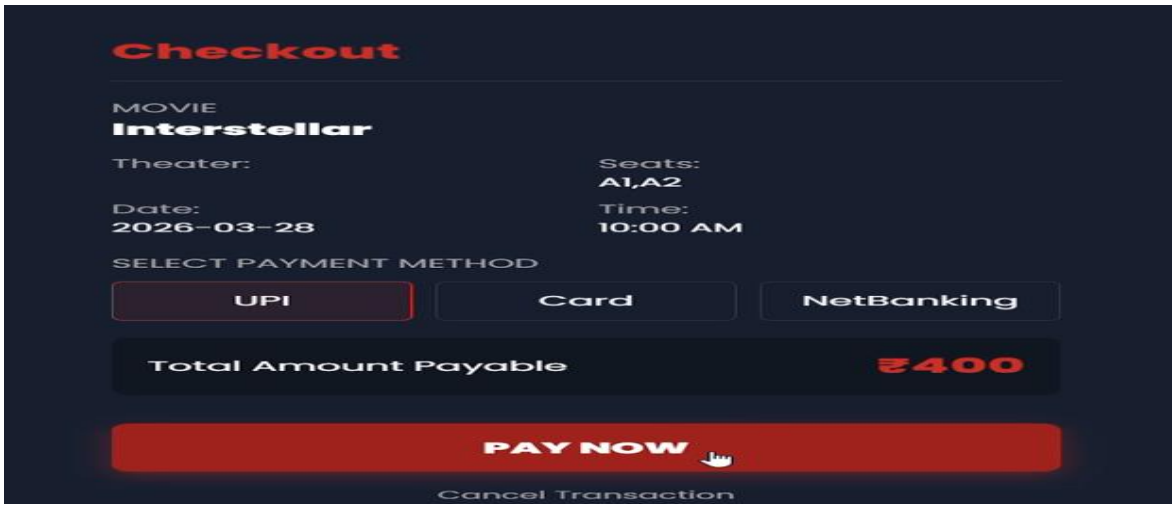
Booking Movie Page:



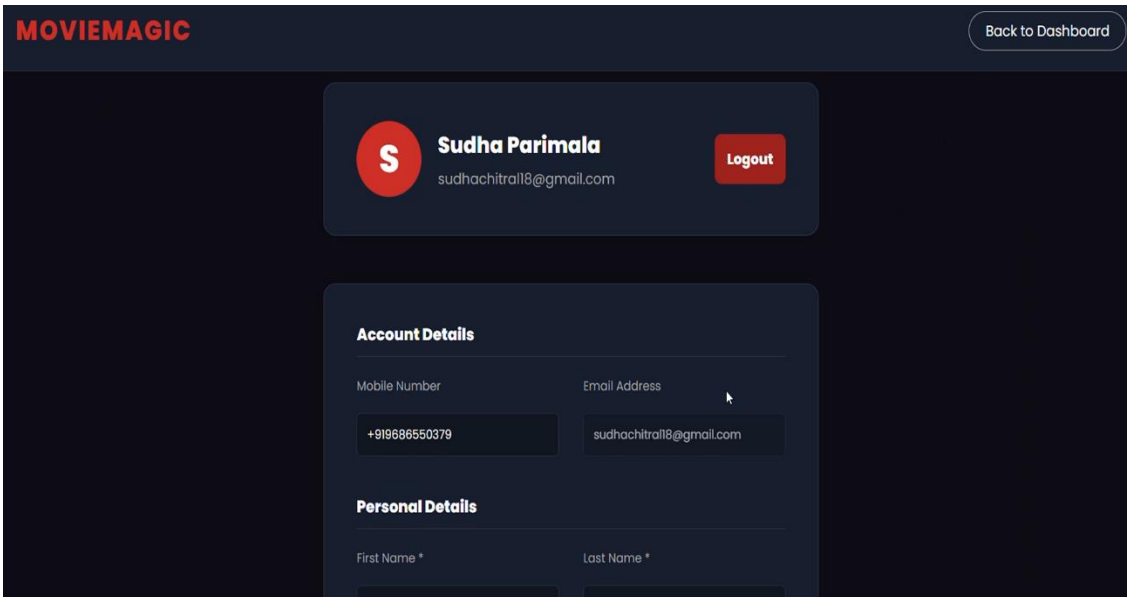




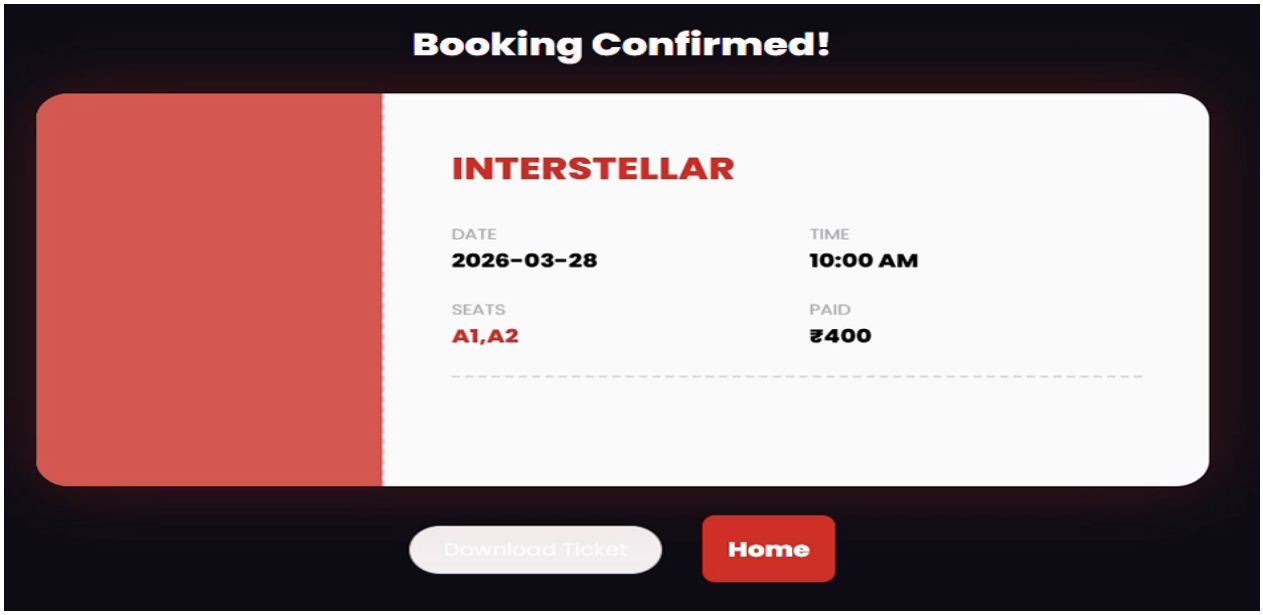
Booking Movie Details:



Profile Page:



Ticket Download page:



Conclusion:

The **Movie Magic movie ticket booking system** has been successfully developed and deployed using a scalable cloud-based architecture. By utilizing AWS services such as **Amazon EC2** for application hosting, **DynamoDB** for storing user, movie, and booking data, and **AWS SNS** for sending booking notifications, the platform provides an efficient and reliable solution for online movie ticket booking.

The system allows users to register, log in, browse available movies, view movie details, and book tickets through a simple and user-friendly interface. The **Flask backend** efficiently manages user authentication, movie information retrieval, booking processes, and profile management while ensuring secure data handling. Additionally, the admin portal enables administrators to add, update, and delete movie records, making the system easy to manage.

Integration with **AWS SNS** enables real-time email notifications for ticket confirmations, improving the overall user experience. In conclusion, Movie Magic demonstrates how cloud technologies and web development frameworks can be combined to build a modern, scalable, and efficient movie ticket booking platform that enhances convenience for both users and administrator